

INM707 Deep Reinforcement Learning Coursework

Notebook 1: Basic Q-learning in a Custom GridWorld

This notebook covers Tasks 1–6 of the coursework. I define a custom finite GridWorld environment, formulate it as a Markov Decision Process, implement tabular Q-learning, evaluate learning performance, and compare different hyperparameter and reward-design choices.

Task 1: Environment and Problem Definition

I define a custom delivery GridWorld environment. The agent starts in a fixed cell, must collect a package from a pickup location, and then deliver it to a goal location. The environment also contains blocked cells, which cannot be entered, and hazard cells, which are legal to enter but produce a penalty.

The problem is to learn a policy that completes the delivery task with high return and as few steps as possible. Since the state space and action space are finite, this environment is suitable for tabular Q-learning.

```
In [7]: from pathlib import Path
import pandas as pd
import random
from dataclasses import dataclass
from typing import Dict, List, Tuple

import numpy as np
import matplotlib.pyplot as plt

# Reproducibility
SEED = 42
random.seed(SEED)
np.random.seed(SEED)

# Save results outside the notebook folder
RESULTS_DIR = Path("../Results/basic")
RESULTS_DIR.mkdir(parents=True, exist_ok=True)

print(f"Results will be saved to: {RESULTS_DIR.resolve()}")
```

Results will be saved to: F:\data sci\INM707-DRL-Coursework\Results\basic

This cell imports the libraries used for the basic Q-learning experiments, fixes the random seed for reproducibility, and creates a results directory for plots and outputs generated by this notebook.

1.1 Defining the GridWorld Layout

The environment is a 6×6 grid. The agent begins at the start cell, must visit the pickup cell, and then terminate the episode by reaching the delivery cell while carrying the package. Blocked cells represent obstacles and cannot be entered. Hazard cells represent undesirable but legal states.

```
In [2]: @dataclass(frozen=True)
class GridWorldConfig:
    rows: int = 6
    cols: int = 6
    start: Tuple[int, int] = (5, 0)
    pickup: Tuple[int, int] = (1, 4)
    delivery: Tuple[int, int] = (0, 5)
    blocked_cells: Tuple[Tuple[int, int], ...] = ((2, 2), (3, 2), (4, 4))
    hazard_cells: Tuple[Tuple[int, int], ...] = ((1, 1), (3, 4))

    # Reward scheme
    step_reward: float = -1.0
    invalid_move_reward: float = -5.0
    hazard_reward: float = -10.0
    pickup_reward: float = 10.0
    delivery_reward: float = 50.0

    max_steps: int = 100

config = GridWorldConfig()
config
```

```
Out[2]: GridWorldConfig(rows=6, cols=6, start=(5, 0), pickup=(1, 4), delivery=(0, 5), blocked_cells=((2, 2), (3, 2), (4, 4)), hazard_cells=((1, 1), (3, 4)), step_reward=-1.0, invalid_move_reward=-5.0, hazard_reward=-10.0, pickup_reward=10.0, delivery_reward=50.0, max_steps=100)
```

The configuration stores the grid dimensions, special cell locations, reward values, and maximum episode length. Keeping these values in one configuration object makes later experiments easier because I can change reward settings without rewriting the environment logic.

1.2 Implementing the GridWorld Environment

The environment follows the usual reinforcement learning interface: `reset()` starts a new episode and `step(action)` applies one action. The state is represented as `(row, col, carrying)`, where `carrying` is `0` before pickup and `1` after pickup. I also encode each state as an integer so it can be used as an index in a Q-table.

```
In [8]: class DeliveryGridWorld:
    """
    Custom deterministic GridWorld environment for tabular Q-learning.

    State:
```

```

        (row, col, carrying)
        carrying = 0 before the package is picked up
        carrying = 1 after the package is picked up

    Actions:
        0 = up
        1 = down
        2 = left
        3 = right
        4 = wait
    """

    ACTIONS: Dict[int, str] = {
        0: "up",
        1: "down",
        2: "left",
        3: "right",
        4: "wait",
    }

    ACTION_DELTAS: Dict[int, Tuple[int, int]] = {
        0: (-1, 0),
        1: (1, 0),
        2: (0, -1),
        3: (0, 1),
        4: (0, 0),
    }

    def __init__(self, config: GridWorldConfig):
        self.config = config
        self.blocked_cells = set(config.blocked_cells)
        self.hazard_cells = set(config.hazard_cells)

        self.valid_positions = [
            (r, c)
            for r in range(config.rows)
            for c in range(config.cols)
            if (r, c) not in self.blocked_cells
        ]

        # Two carrying states: 0 = not carrying, 1 = carrying
        self.states = [
            (r, c, carrying)
            for (r, c) in self.valid_positions
            for carrying in (0, 1)
        ]

        self.state_to_id = {state: i for i, state in enumerate(self.states)}
        self.id_to_state = {i: state for state, i in self.state_to_id.items()}

        self.n_states = len(self.states)
        self.n_actions = len(self.ACTIONS)

        self.reset()

    def reset(self) -> int:
        """Reset the environment to the initial state and return the encoded state id"""
        self.row, self.col = self.config.start
        self.carrying = 0
        self.steps = 0

```

```

        self.done = False
        return self.encode_state((self.row, self.col, self.carrying))

    def encode_state(self, state: Tuple[int, int, int]) -> int:
        """Convert a tuple state into an integer state ID."""
        return self.state_to_id[state]

    def decode_state(self, state_id: int) -> Tuple[int, int, int]:
        """Convert an integer state ID back into a tuple state."""
        return self.id_to_state[state_id]

    def is_inside_grid(self, row: int, col: int) -> bool:
        """Check whether a position lies inside the grid boundaries."""
        return 0 <= row < self.config.rows and 0 <= col < self.config.cols

    def is_valid_position(self, row: int, col: int) -> bool:
        """Check whether a position is inside the grid and not blocked."""
        return self.is_inside_grid(row, col) and (row, col) not in self.blocked_

    def step(self, action: int) -> Tuple[int, float, bool, Dict]:
        """
        Apply one action and return:
        next_state_id, reward, done, info
        """
        if self.done:
            raise ValueError("Episode has already ended. Call reset() before tak

        if action not in self.ACTIONS:
            raise ValueError(f"Unknown action: {action}")

        self.steps += 1
        reward = self.config.step_reward
        info = {"event": "step"}

        dr, dc = self.ACTION_DELTAS[action]
        next_row = self.row + dr
        next_col = self.col + dc

        # Invalid movement leaves the agent in the same state.
        if not self.is_valid_position(next_row, next_col):
            next_row, next_col = self.row, self.col
            reward = self.config.invalid_move_reward
            info["event"] = "invalid_move"

        # Valid movement updates the agent position.
        self.row, self.col = next_row, next_col

        # Hazard penalty is added when the agent enters a hazard cell.
        if (self.row, self.col) in self.hazard_cells:
            reward += self.config.hazard_reward
            info["event"] = "hazard"

        # Pickup happens automatically when the agent reaches the pickup cell.
        if (self.row, self.col) == self.config.pickup and self.carrying == 0:
            self.carrying = 1
            reward += self.config.pickup_reward
            info["event"] = "pickup"

        # Delivery terminates the episode only if the package has been collected
        if (self.row, self.col) == self.config.delivery and self.carrying == 1:

```

```

        reward += self.config.delivery_reward
        self.done = True
        info["event"] = "delivery"

    # Safety termination to prevent infinite episodes.
    if self.steps >= self.config.max_steps:
        self.done = True
        info["event"] = "max_steps"

    next_state_id = self.encode_state((self.row, self.col, self.carrying))
    return next_state_id, reward, self.done, info

def render_text(self) -> None:
    """Print a simple text representation of the current environment state."""
    grid = [["." for _ in range(self.config.cols)] for _ in range(self.config.rows)]

    for r, c in self.blocked_cells:
        grid[r][c] = "X"

    for r, c in self.hazard_cells:
        grid[r][c] = "H"

    pr, pc = self.config.pickup
    dr, dc = self.config.delivery
    grid[pr][pc] = "P"
    grid[dr][dc] = "D"

    grid[self.row][self.col] = "A" if self.carrying == 0 else "C"

    for row in grid:
        print(" ".join(row))

    print(f"\nA = agent, C = carrying package, P = pickup, D = delivery, X = blocked, H = hazard")
    print(f"Current state: row={self.row}, col={self.col}, carrying={self.carrying}")

```

The environment is deterministic: each action maps to a predictable next state unless the move is invalid, in which case the agent remains in place and receives a penalty. The package is picked up automatically at the pickup cell, and the episode ends only when the agent reaches the delivery cell while carrying the package.

```

In [9]: env = DeliveryGridWorld(config)

print(f"Number of valid grid positions: {len(env.valid_positions)}")
print(f"Number of states: {env.n_states}")
print(f"Number of actions: {env.n_actions}")
print(f"Initial encoded state: {env.reset()}")
print(f"Initial decoded state: {env.decode_state(env.reset())}\n")

env.render_text()

```

Number of valid grid positions: 33
 Number of states: 66
 Number of actions: 5
 Initial encoded state: 54
 Initial decoded state: (5, 0, 0)

```
. . . . . D
. H . . P .
. . X . . .
. . X . H .
. . . . X .
A . . . . .
```

A = agent, C = carrying package, P = pickup, D = delivery, X = blocked, H = hazard

Current state: row=5, col=0, carrying=0, steps=0

The state space contains one state for each valid grid position and carrying status. Since there are blocked cells, the number of valid positions is smaller than the full 6×6 grid.

The action space contains five discrete actions.

Task 2: State Transition Function and Reward Function

The environment can be represented as a finite Markov Decision Process. The transition function is deterministic because the next state is fully determined by the current state and selected action. The reward function gives a step cost to encourage short paths, penalties for invalid or hazardous movement, and positive rewards for pickup and delivery.

2.1 Formal MDP Components

I define the MDP as follows:

- State space: ($S = \{(r, c, b)\}$), where (r) is the row, (c) is the column, and ($b \in \{0,1\}$) indicates whether the agent is carrying the package.
- Action space: ($A = \{\text{up}, \text{down}, \text{left}, \text{right}, \text{wait}\}$).
- Transition function: ($s' = T(s,a)$). Movement actions change the agent position unless the target cell is outside the grid or blocked. Invalid moves leave the agent in the same state.
- Reward function: ($R(s,a,s')$). The reward depends on movement cost, invalid movement, hazard cells, pickup, and successful delivery.
- Terminal condition: the episode terminates when the agent reaches the delivery cell while carrying the package, or when the maximum step limit is reached.

```
In [10]: reward_summary = {
    "Event": [
        "Valid movement / wait",
        "Invalid movement into wall, boundary or blocked cell",
        "Entering a hazard cell",
```

```

        "First arrival at pickup cell",
        "Successful delivery while carrying package",
        "Maximum episode length reached",
    ],
    "Reward contribution": [
        config.step_reward,
        config.invalid_move_reward,
        config.hazard_reward,
        config.pickup_reward,
        config.delivery_reward,
        "No extra reward; episode terminates",
    ],
}

reward_summary

```

```

Out[10]: {'Event': ['Valid movement / wait',
  'Invalid movement into wall, boundary or blocked cell',
  'Entering a hazard cell',
  'First arrival at pickup cell',
  'Successful delivery while carrying package',
  'Maximum episode length reached'],
  'Reward contribution': [-1.0,
    -5.0,
    -10.0,
    10.0,
    50.0,
    'No extra reward; episode terminates']}

```

The reward structure combines sparse and shaped components. The delivery reward is the main task objective, while the step cost and penalties guide the agent toward shorter and safer routes.

2.2 Manual Transition Test

Before training, I test the environment manually by applying a sequence of actions. This verifies that the transition function, pickup rule, delivery rule and reward function behave as intended.

```

In [13]: def run_manual_actions(env: DeliveryGridWorld, actions: List[int]) -> List[Dict]
    """
    Run a fixed action sequence and record the resulting transition information.
    """
    state = env.reset()
    history = []

    for action in actions:
        next_state, reward, done, info = env.step(action)

        history.append({
            "state": env.decode_state(state),
            "action": env.ACTIONS[action],
            "next_state": env.decode_state(next_state),
            "reward": reward,
            "event": info["event"],
            "done": done,
        })

```

```

        state = next_state

        if done:
            break

    return history

# A hand-designed safe route from start -> pickup -> delivery.
# From (5,0) to (1,4), avoiding the hazard at (1,1),
# then from (1,4) to (0,5).
manual_actions = [
    3, 3, 3, 3, 3, # move right along the bottom row to (5,5)
    0, 0, 0, 0,    # move up to (1,5)
    2,             # move left to pickup at (1,4)
    0,             # move up to (0,4)
    3              # move right to delivery at (0,5)
]

env = DeliveryGridWorld(config)
manual_history = run_manual_actions(env, manual_actions)

pd.DataFrame(manual_history)

```

Out[13]:

	state	action	next_state	reward	event	done
0	(5, 0, 0)	right	(5, 1, 0)	-1.0	step	False
1	(5, 1, 0)	right	(5, 2, 0)	-1.0	step	False
2	(5, 2, 0)	right	(5, 3, 0)	-1.0	step	False
3	(5, 3, 0)	right	(5, 4, 0)	-1.0	step	False
4	(5, 4, 0)	right	(5, 5, 0)	-1.0	step	False
5	(5, 5, 0)	up	(4, 5, 0)	-1.0	step	False
6	(4, 5, 0)	up	(3, 5, 0)	-1.0	step	False
7	(3, 5, 0)	up	(2, 5, 0)	-1.0	step	False
8	(2, 5, 0)	up	(1, 5, 0)	-1.0	step	False
9	(1, 5, 0)	left	(1, 4, 1)	9.0	pickup	False
10	(1, 4, 1)	up	(0, 4, 1)	-1.0	step	False
11	(0, 4, 1)	right	(0, 5, 1)	49.0	delivery	True

The manual rollout follows a feasible route from the start cell to the pickup cell and then to the delivery cell. This test confirms that pickup changes the carrying variable from 0 to 1, and that delivery terminates the episode only after the package has been collected.

In [14]: `env.render_text()`

```

. . . . . C
. H . . P .
. . X . . .
. . X . H .
. . . . X .
. . . . .

```

A = agent, C = carrying package, P = pickup, D = delivery, X = blocked, H = hazard

Current state: row=0, col=5, carrying=1, steps=12

The rendered final state shows the agent at the delivery cell while carrying the package.
This is the desired terminal condition for the task.

Task 3: Q-learning Parameters and Policy

I use tabular Q-learning to learn an action-value function ($Q(s,a)$). Since the environment has a finite state space and discrete action space, the Q-function can be represented directly as a table with one row per state and one column per action.

The Q-learning update rule is:

$$[Q(s,a) \leftarrow Q(s,a) + \alpha \left[r + \gamma \max_{a'} Q(s',a') - Q(s,a) \right]]$$

where (α) is the learning rate and (γ) is the discount factor. Action selection is controlled using an epsilon-greedy policy, where the agent explores with probability (ϵ) and otherwise exploits the action with the highest current Q-value.

```

In [15]: def choose_epsilon_greedy_action(q_table: np.ndarray, state: int, epsilon: float)
         """
         Select an action using an epsilon-greedy policy.

         With probability epsilon, the agent explores by choosing a random action.
         Otherwise, it exploits by choosing one of the actions with the highest Q-val
         Random tie-breaking avoids always selecting the lowest-index action when Q-v
         """
         if np.random.random() < epsilon:
             return np.random.randint(n_actions)

         q_values = q_table[state]
         best_actions = np.flatnonzero(q_values == np.max(q_values))
         return int(np.random.choice(best_actions))

def evaluate_greedy_policy(
    env: DeliveryGridWorld,
    q_table: np.ndarray,
    episodes: int = 20
) -> Dict[str, float]:
    """
    Evaluate the current greedy policy without exploration.
    """
    returns = []
    steps = []
    successes = []

```

```

for _ in range(episodes):
    state = env.reset()
    total_reward = 0.0

    for _ in range(env.config.max_steps):
        action = int(np.argmax(q_table[state]))
        next_state, reward, done, info = env.step(action)

        total_reward += reward
        state = next_state

        if done:
            successes.append(info["event"] == "delivery")
            break

    returns.append(total_reward)
    steps.append(env.steps)

return {
    "mean_return": float(np.mean(returns)),
    "mean_steps": float(np.mean(steps)),
    "success_rate": float(np.mean(successes)) if successes else 0.0,
}

```

The action-selection function implements epsilon-greedy exploration with random tie-breaking. The evaluation function runs the learned policy greedily, without exploration, so that training performance and final policy quality can be measured separately.

3.1 Q-learning Training Function

The training function runs repeated episodes of interaction with the environment. At each step, the agent selects an action using the epsilon-greedy policy, observes the next state and reward, and updates the Q-table. Epsilon decays after each episode so that the agent explores heavily early in training and increasingly exploits learned values later.

```

In [16]: def train_q_learning(
    env: DeliveryGridWorld,
    episodes: int = 2000,
    alpha: float = 0.5,
    gamma: float = 0.95,
    epsilon_start: float = 1.0,
    epsilon_min: float = 0.05,
    epsilon_decay: float = 0.995,
    eval_every: int = 50,
    eval_episodes: int = 20,
) -> Tuple[np.ndarray, pd.DataFrame]:
    """
    Train a tabular Q-learning agent in the DeliveryGridWorld environment.

    Returns:
        q_table: learned action-value table
        history_df: training and evaluation metrics recorded over time
    """
    q_table = np.zeros((env.n_states, env.n_actions), dtype=np.float64)
    epsilon = epsilon_start

```

```

history = []

for episode in range(1, episodes + 1):
    state = env.reset()
    total_reward = 0.0
    done = False

    while not done:
        action = choose_epsilon_greedy_action(
            q_table=q_table,
            state=state,
            epsilon=epsilon,
            n_actions=env.n_actions,
        )

        next_state, reward, done, info = env.step(action)

        best_next_q = np.max(q_table[next_state])
        td_target = reward + gamma * best_next_q * (not done)
        td_error = td_target - q_table[state, action]
        q_table[state, action] += alpha * td_error

        total_reward += reward
        state = next_state

    if episode % eval_every == 0 or episode == 1:
        evaluation = evaluate_greedy_policy(env, q_table, episodes=eval_episodes)
    else:
        evaluation = {
            "mean_return": np.nan,
            "mean_steps": np.nan,
            "success_rate": np.nan,
        }

    history.append({
        "episode": episode,
        "training_return": total_reward,
        "training_steps": env.steps,
        "epsilon": epsilon,
        "eval_mean_return": evaluation["mean_return"],
        "eval_mean_steps": evaluation["mean_steps"],
        "eval_success_rate": evaluation["success_rate"],
    })

    epsilon = max(epsilon_min, epsilon * epsilon_decay)

history_df = pd.DataFrame(history)
return q_table, history_df

```

The training loop records both training metrics and periodic greedy-policy evaluation metrics. This distinction is important because training episodes include exploration, whereas evaluation episodes measure the quality of the learned policy without random exploratory actions.

3.2 Baseline Hyperparameters

I use a baseline configuration before running the wider parameter experiments. The baseline uses a moderate learning rate, a high discount factor, and gradual epsilon decay. This allows the agent to explore early episodes while still converging toward a greedy policy later in training.

```
In [17]: baseline_params = {  
    "episodes": 2000,  
    "alpha": 0.5,  
    "gamma": 0.95,  
    "epsilon_start": 1.0,  
    "epsilon_min": 0.05,  
    "epsilon_decay": 0.995,  
    "eval_every": 50,  
    "eval_episodes": 20,  
}  
  
baseline_params
```

```
Out[17]: {'episodes': 2000,  
    'alpha': 0.5,  
    'gamma': 0.95,  
    'epsilon_start': 1.0,  
    'epsilon_min': 0.05,  
    'epsilon_decay': 0.995,  
    'eval_every': 50,  
    'eval_episodes': 20}
```

These values are used as the initial Q-learning configuration. Later experiments vary the main hyperparameters to analyse how learning rate, discount factor and exploration schedule affect convergence.

Task 4: Running Q-learning and Representing Performance

I now train the baseline Q-learning agent using the parameters defined above. Performance is represented using episode return, episode length, epsilon decay, and periodic greedy-policy evaluation. A successful learned policy should achieve a high return, complete the task in a small number of steps, and obtain a high evaluation success rate.

```
In [18]: env = DeliveryGridWorld(config)  
  
baseline_q_table, baseline_history = train_q_learning(  
    env=env,  
    **baseline_params  
)  
  
baseline_history.tail()
```

Out[18]:

	episode	training_return	training_steps	epsilon	eval_mean_return	eval_mean_ste
1995	1996	50.0	10	0.05	NaN	N
1996	1997	50.0	10	0.05	NaN	N
1997	1998	48.0	12	0.05	NaN	N
1998	1999	50.0	10	0.05	NaN	N
1999	2000	50.0	10	0.05	50.0	10

The final rows of the training history show the agent's late-training performance. At this stage, epsilon has decayed close to its minimum value, so most actions are selected greedily with limited exploration.

```
In [19]: baseline_eval = evaluate_greedy_policy(
    env=DeliveryGridWorld(config),
    q_table=baseline_q_table,
    episodes=100
)

baseline_eval
```

```
Out[19]: {'mean_return': 50.0, 'mean_steps': 10.0, 'success_rate': 1.0}
```

The greedy-policy evaluation measures the learned policy without exploratory actions. This gives a clearer estimate of the final policy quality than the raw training return, because training episodes still include occasional random exploration.

4.1 Baseline Learning Curves

I plot the training return and episode length using a rolling average to make the learning trend easier to interpret. I also plot the periodic greedy-policy evaluation metrics to show whether the learned policy solves the delivery task reliably.

```
In [20]: def add_rolling_metrics(history_df: pd.DataFrame, window: int = 50) -> pd.DataFrame:
    """
    Add rolling averages for smoother visualisation of training performance.
    """
    df = history_df.copy()
    df["rolling_return"] = df["training_return"].rolling(window=window, min_periods=1)
    df["rolling_steps"] = df["training_steps"].rolling(window=window, min_periods=1)
    return df

baseline_history_plot = add_rolling_metrics(baseline_history, window=50)
baseline_history_plot.head()
```

Out[20]:

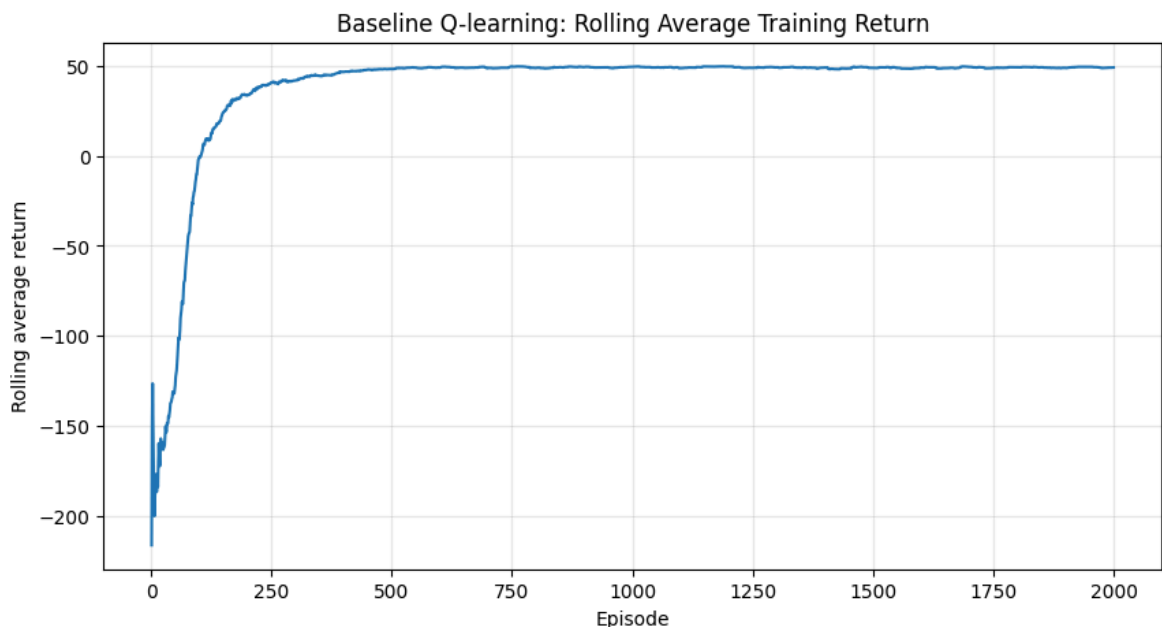
	episode	training_return	training_steps	epsilon	eval_mean_return	eval_mean_steps
0	1	-216.0	100	1.000000	-480.0	100.0
1	2	-84.0	60	0.995000	NaN	NaN
2	3	-79.0	79	0.990025	NaN	NaN
3	4	-210.0	100	0.985075	NaN	NaN
4	5	-240.0	100	0.980150	NaN	NaN

Rolling averages reduce the visual noise caused by exploration. I use a 50-episode window because it is large enough to show the trend while still preserving the shape of the learning process.

```
In [21]: plt.figure(figsize=(10, 5))
plt.plot(baseline_history_plot["episode"], baseline_history_plot["rolling_return"])
plt.xlabel("Episode")
plt.ylabel("Rolling average return")
plt.title("Baseline Q-learning: Rolling Average Training Return")
plt.grid(True, alpha=0.3)

return_plot_path = RESULTS_DIR / "baseline_q_learning_rolling_return.png"
plt.savefig(return_plot_path, dpi=300, bbox_inches="tight")
plt.show()

print(f"Saved plot to: {return_plot_path}")
```



Saved plot to: ..\Results\basic\baseline_q_learning_rolling_return.png

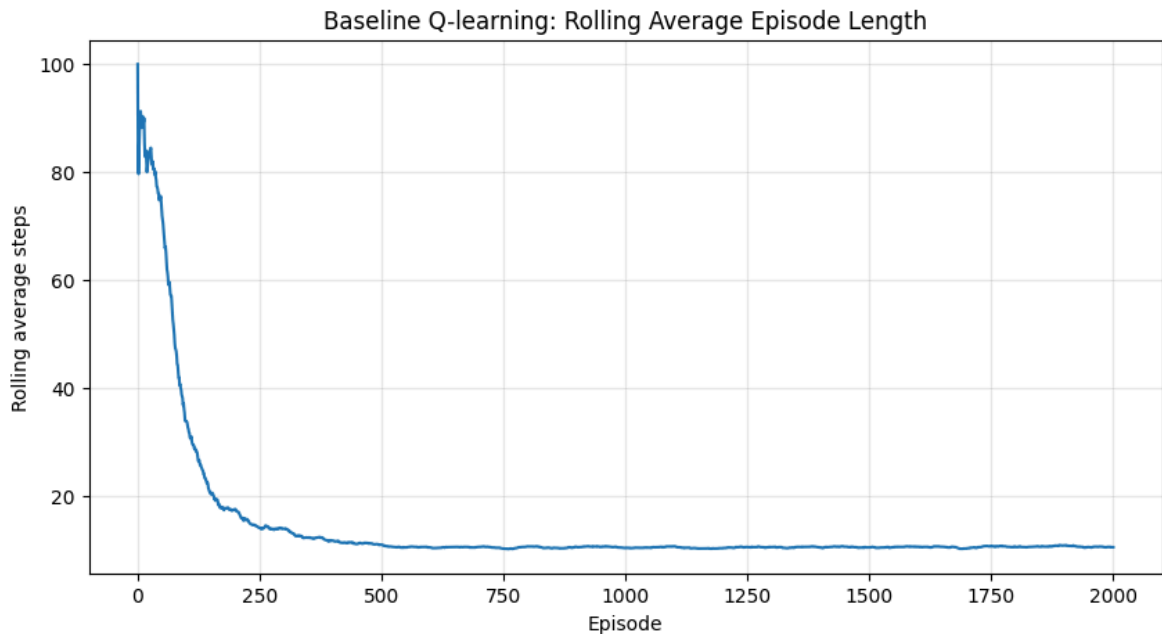
The rolling return curve shows whether the agent improves its cumulative reward over training. Increasing return indicates that the Q-table is being updated toward actions that complete the task more efficiently.

```
In [22]: plt.figure(figsize=(10, 5))
plt.plot(baseline_history_plot["episode"], baseline_history_plot["rolling_steps"])
plt.xlabel("Episode")
```

```
plt.ylabel("Rolling average steps")
plt.title("Baseline Q-learning: Rolling Average Episode Length")
plt.grid(True, alpha=0.3)

steps_plot_path = RESULTS_DIR / "baseline_q_learning_rolling_steps.png"
plt.savefig(steps_plot_path, dpi=300, bbox_inches="tight")
plt.show()

print(f"Saved plot to: {steps_plot_path}")
```



Saved plot to: ..\Results\basic\baseline_q_learning_rolling_steps.png

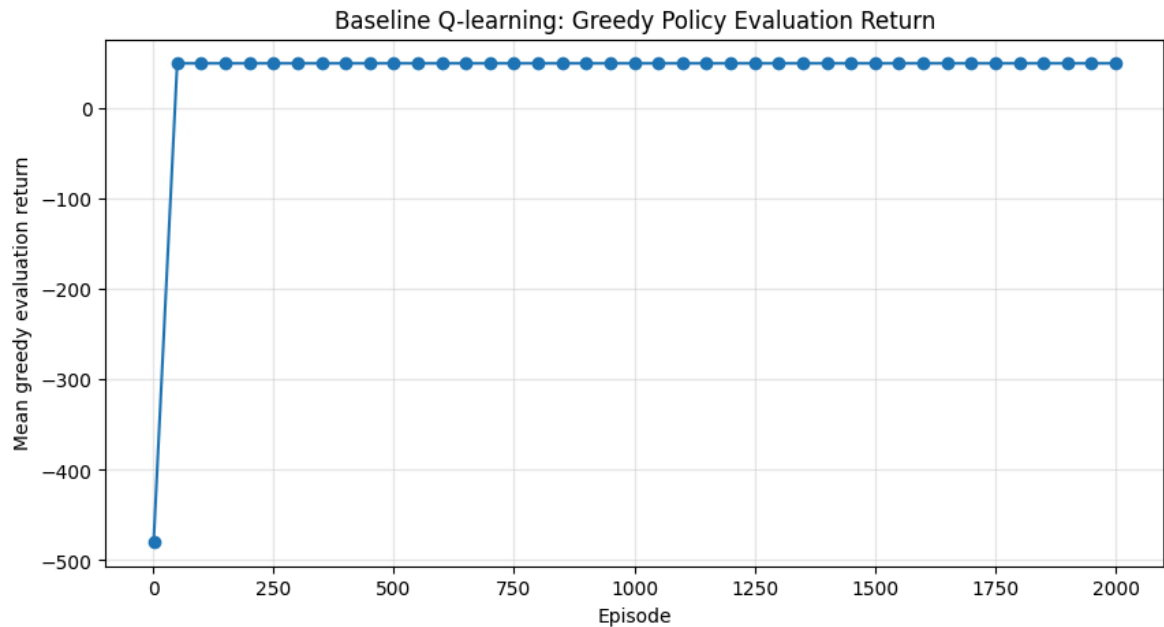
The rolling episode-length curve shows how quickly the agent completes the task. A decreasing curve indicates that the learned policy is becoming more direct and efficient.

```
In [23]: eval_points = baseline_history_plot.dropna(subset=["eval_mean_return", "eval_mean_episode_length"])

plt.figure(figsize=(10, 5))
plt.plot(eval_points["episode"], eval_points["eval_mean_return"], marker="o")
plt.xlabel("Episode")
plt.ylabel("Mean greedy evaluation return")
plt.title("Baseline Q-learning: Greedy Policy Evaluation Return")
plt.grid(True, alpha=0.3)

eval_return_plot_path = RESULTS_DIR / "baseline_q_learning_eval_return.png"
plt.savefig(eval_return_plot_path, dpi=300, bbox_inches="tight")
plt.show()

print(f"Saved plot to: {eval_return_plot_path}")
```



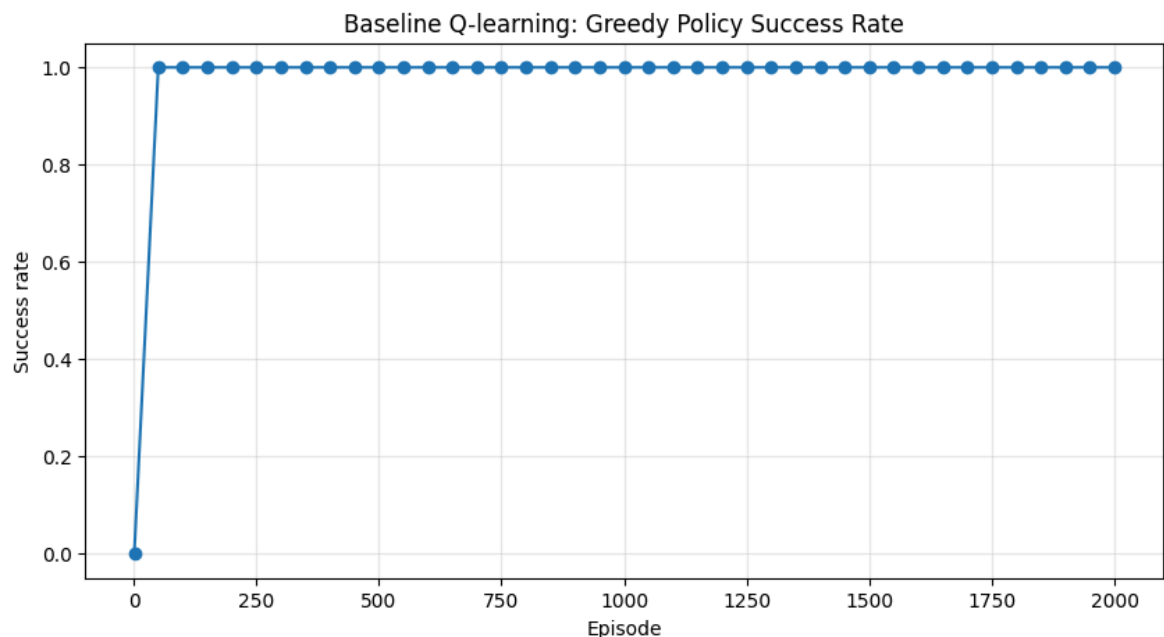
Saved plot to: ..\Results\basic\baseline_q_learning_eval_return.png

The greedy evaluation return measures the quality of the current learned policy at fixed intervals during training. Unlike the training return, this curve is not directly affected by exploratory random actions.

```
In [24]: plt.figure(figsize=(10, 5))
plt.plot(eval_points["episode"], eval_points["eval_success_rate"], marker="o")
plt.xlabel("Episode")
plt.ylabel("Success rate")
plt.ylim(-0.05, 1.05)
plt.title("Baseline Q-learning: Greedy Policy Success Rate")
plt.grid(True, alpha=0.3)

success_plot_path = RESULTS_DIR / "baseline_q_learning_success_rate.png"
plt.savefig(success_plot_path, dpi=300, bbox_inches="tight")
plt.show()

print(f"Saved plot to: {success_plot_path}")
```



Saved plot to: ..\Results\basic\baseline_q_learning_success_rate.png

The success-rate curve shows how often the greedy policy completes the pickup-and-delivery task. A success rate of 1.0 means that the learned policy solves the task in every evaluation episode.

4.2 Baseline Performance Summary

To support the plots with numerical evidence, I summarise the final training performance and the final greedy-policy evaluation. The final training values are calculated from the last 100 episodes, while the greedy-policy evaluation is calculated over 100 evaluation episodes with no exploration.

```
In [25]: final_training_summary = {
    "mean_training_return_last_100": baseline_history["training_return"].tail(100),
    "mean_training_steps_last_100": baseline_history["training_steps"].tail(100),
    "final_epsilon": baseline_history["epsilon"].iloc[-1],
    "greedy_eval_mean_return": baseline_eval["mean_return"],
    "greedy_eval_mean_steps": baseline_eval["mean_steps"],
    "greedy_eval_success_rate": baseline_eval["success_rate"],
}

final_training_summary_df = pd.DataFrame([final_training_summary]).round(3)
final_training_summary_df
```

```
Out[25]:
```

	mean_training_return_last_100	mean_training_steps_last_100	final_epsilon	greedy_eval
0	49.13	10.51	0.05	

The baseline agent learns a reliable policy for the delivery task. The final greedy evaluation is the most important performance measure because it tests the learned policy without exploratory actions. A success rate of 1.0 indicates that the agent consistently completes the task after training.

4.3 Learned Greedy Policy Rollout

I inspect the learned greedy policy by running one episode without exploration and recording the state, action, reward and event at each step. This shows whether the final Q-table produces a sensible route through the environment.

```
In [26]: def rollout_greedy_policy(env: DeliveryGridWorld, q_table: np.ndarray) -> pd.DataFrame:
    """
    Run one greedy-policy episode and return a transition table.
    """
    state = env.reset()
    history = []

    for _ in range(env.config.max_steps):
        action = int(np.argmax(q_table[state]))
        next_state, reward, done, info = env.step(action)

        history.append({
```

```

        "state": env.decode_state(state),
        "action": env.ACTIONS[action],
        "next_state": env.decode_state(next_state),
        "reward": reward,
        "event": info["event"],
        "done": done,
    })

    state = next_state

    if done:
        break

    return pd.DataFrame(history)

greedy_rollout_df = rollout_greedy_policy(
    env=DeliveryGridWorld(config),
    q_table=baseline_q_table
)

greedy_rollout_df

```

Out[26]:

	state	action	next_state	reward	event	done
0	(5, 0, 0)	up	(4, 0, 0)	-1.0	step	False
1	(4, 0, 0)	right	(4, 1, 0)	-1.0	step	False
2	(4, 1, 0)	right	(4, 2, 0)	-1.0	step	False
3	(4, 2, 0)	right	(4, 3, 0)	-1.0	step	False
4	(4, 3, 0)	up	(3, 3, 0)	-1.0	step	False
5	(3, 3, 0)	up	(2, 3, 0)	-1.0	step	False
6	(2, 3, 0)	up	(1, 3, 0)	-1.0	step	False
7	(1, 3, 0)	right	(1, 4, 1)	9.0	pickup	False
8	(1, 4, 1)	up	(0, 4, 1)	-1.0	step	False
9	(0, 4, 1)	right	(0, 5, 1)	49.0	delivery	True

The greedy rollout gives an interpretable view of the learned policy. The agent should first navigate to the pickup cell, then move to the delivery cell while carrying the package. The number of rows in this table is the number of actions taken by the final policy.

```

In [27]: greedy_rollout_summary = {
    "total_return": greedy_rollout_df["reward"].sum(),
    "total_steps": len(greedy_rollout_df),
    "completed_delivery": bool(greedy_rollout_df["event"].iloc[-1] == "delivery")
}

pd.DataFrame([greedy_rollout_summary])

```

Out[27]:

	total_return	total_steps	completed_delivery
0	50.0	10	True

This summary confirms the return, path length and completion status of the learned greedy policy. It is useful for checking that the learning curves correspond to an actual successful route rather than only a numerical improvement in the Q-table.

4.4 Interpretation of Baseline Q-learning Performance

The baseline Q-learning agent successfully learns the delivery task. In the final 100 training episodes, the agent achieves a mean training return of approximately 49.13 and a mean episode length of approximately 10.51 steps. The greedy evaluation performs even more cleanly, achieving a mean return of 50.0, a mean episode length of 10.0 steps, and a success rate of 1.0.

The difference between training and greedy evaluation is expected because training still includes limited exploration while epsilon is at its minimum value of 0.05. The greedy rollout confirms that the learned policy is valid and efficient: the agent moves from the start cell to the pickup cell, collects the package, and then reaches the delivery cell in 10 steps. This supports the conclusion that the Q-table has converged to a high-quality policy for this environment.

Task 5: Repeating the Experiment with Different Parameter Values and Policies

I repeat the Q-learning experiment with different hyperparameter and reward-design settings. The purpose is to evaluate how learning rate, discount factor, exploration schedule and reward shaping affect convergence speed, final return and policy reliability.

For each setting, I train a new Q-learning agent and evaluate the final greedy policy over 100 episodes. The main comparison metrics are:

- mean training return over the final 100 episodes,
- mean training steps over the final 100 episodes,
- final greedy evaluation return,
- final greedy evaluation steps,
- final greedy evaluation success rate.

```
In [28]: def run_q_learning_experiment(
    experiment_name: str,
    config: GridWorldConfig,
    episodes: int = 2000,
    alpha: float = 0.5,
    gamma: float = 0.95,
    epsilon_start: float = 1.0,
```

```

epsilon_min: float = 0.05,
epsilon_decay: float = 0.995,
eval_every: int = 50,
eval_episodes: int = 20,
final_eval_episodes: int = 100,
) -> Tuple[np.ndarray, pd.DataFrame, Dict]:
    """
    Train and evaluate one Q-learning configuration.
    """
    env = DeliveryGridWorld(config)

    q_table, history = train_q_learning(
        env=env,
        episodes=episodes,
        alpha=alpha,
        gamma=gamma,
        epsilon_start=epsilon_start,
        epsilon_min=epsilon_min,
        epsilon_decay=epsilon_decay,
        eval_every=eval_every,
        eval_episodes=eval_episodes,
    )

    final_eval = evaluate_greedy_policy(
        env=DeliveryGridWorld(config),
        q_table=q_table,
        episodes=final_eval_episodes,
    )

    summary = {
        "experiment": experiment_name,
        "alpha": alpha,
        "gamma": gamma,
        "epsilon_decay": epsilon_decay,
        "step_reward": config.step_reward,
        "invalid_move_reward": config.invalid_move_reward,
        "hazard_reward": config.hazard_reward,
        "pickup_reward": config.pickup_reward,
        "delivery_reward": config.delivery_reward,
        "mean_training_return_last_100": history["training_return"].tail(100).mean,
        "mean_training_steps_last_100": history["training_steps"].tail(100).mean,
        "final_epsilon": history["epsilon"].iloc[-1],
        "greedy_eval_mean_return": final_eval["mean_return"],
        "greedy_eval_mean_steps": final_eval["mean_steps"],
        "greedy_eval_success_rate": final_eval["success_rate"],
    }

    return q_table, history, summary

```

This helper function makes the experiment process consistent. Each configuration is trained from a fresh Q-table and evaluated using the same final greedy-policy procedure, so the results are directly comparable.

5.1 Learning Rate Experiments

The learning rate (α) controls how strongly new temporal-difference updates overwrite existing Q-values. A small value learns cautiously, while a large value updates

more aggressively. I compare low, medium and high learning rates while keeping the other parameters fixed.

```
In [29]: alpha_experiments = [
    ("alpha_0.1", 0.1),
    ("alpha_0.5", 0.5),
    ("alpha_0.9", 0.9),
]

alpha_results = []

for name, alpha_value in alpha_experiments:
    _, history, summary = run_q_learning_experiment(
        experiment_name=name,
        config=config,
        episodes=2000,
        alpha=alpha_value,
        gamma=0.95,
        epsilon_start=1.0,
        epsilon_min=0.05,
        epsilon_decay=0.995,
        eval_every=50,
        eval_episodes=20,
    )

    history = add_rolling_metrics(history, window=50)
    history["experiment"] = name

    alpha_results.append((history, summary))

alpha_summary_df = pd.DataFrame([summary for _, summary in alpha_results]).round
alpha_summary_df
```

```
Out[29]:
```

	experiment	alpha	gamma	epsilon_decay	step_reward	invalid_move_reward	hazard
0	alpha_0.1	0.1	0.95	0.995	-1.0	-5.0	
1	alpha_0.5	0.5	0.95	0.995	-1.0	-5.0	
2	alpha_0.9	0.9	0.95	0.995	-1.0	-5.0	

The learning-rate table compares the final performance of each alpha value. The best setting should combine high final return, low episode length and a greedy evaluation success rate of 1.0.

```
In [30]: plt.figure(figsize=(10, 5))

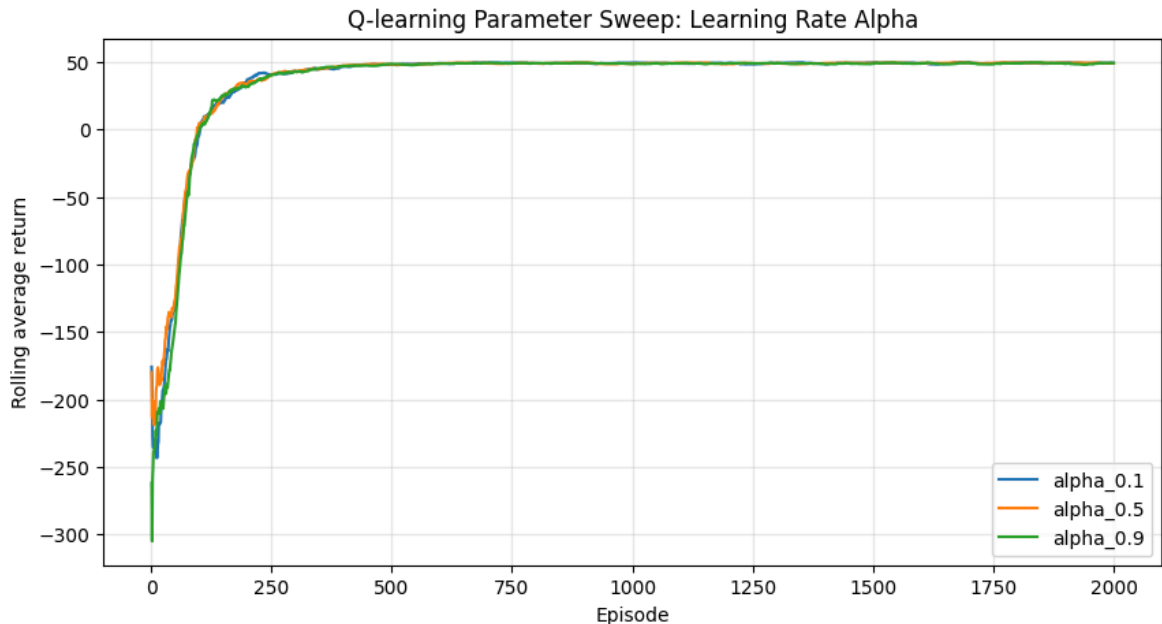
for history, _ in alpha_results:
    plt.plot(
        history["episode"],
        history["rolling_return"],
        label=history["experiment"].iloc[0],
    )

plt.xlabel("Episode")
plt.ylabel("Rolling average return")
plt.title("Q-learning Parameter Sweep: Learning Rate Alpha")
```

```
plt.legend()
plt.grid(True, alpha=0.3)

alpha_plot_path = RESULTS_DIR / "parameter_sweep_alpha.png"
plt.savefig(alpha_plot_path, dpi=300, bbox_inches="tight")
plt.show()

print(f"Saved plot to: {alpha_plot_path}")
```



Saved plot to: ..\Results\basic\parameter_sweep_alpha.png

The alpha comparison shows how quickly each learning-rate setting improves its return. Faster improvement indicates quicker propagation of useful Q-value updates, while unstable curves would suggest overly aggressive updates.

5.1.1 Interpretation of Learning Rate Results

All three learning-rate settings learn a successful greedy policy with a final evaluation success rate of 1.0 and a mean greedy evaluation return of 50.0. This indicates that the task is learnable across a wide range of alpha values. The final policy is therefore not highly sensitive to the learning rate in this small deterministic environment.

The main difference appears in the early training phase rather than in the final result. Since the state space is small and the reward function is shaped, useful Q-value updates propagate quickly even with a low learning rate. This explains why all three alpha settings eventually converge to the same greedy policy.

5.2 Discount Factor Experiments

The discount factor (γ) controls how much the agent values future rewards. A low gamma makes the agent more short-sighted, while a high gamma places more value on delayed rewards. This is important in the delivery task because the largest reward is only received after the agent first reaches the pickup cell and then reaches the delivery cell.

```

In [31]: gamma_experiments = [
    ("gamma_0.50", 0.50),
    ("gamma_0.75", 0.75),
    ("gamma_0.95", 0.95),
    ("gamma_0.99", 0.99),
]

gamma_results = []

for name, gamma_value in gamma_experiments:
    _, history, summary = run_q_learning_experiment(
        experiment_name=name,
        config=config,
        episodes=2000,
        alpha=0.5,
        gamma=gamma_value,
        epsilon_start=1.0,
        epsilon_min=0.05,
        epsilon_decay=0.995,
        eval_every=50,
        eval_episodes=20,
    )

    history = add_rolling_metrics(history, window=50)
    history["experiment"] = name

    gamma_results.append((history, summary))

gamma_summary_df = pd.DataFrame([summary for _, summary in gamma_results]).round
gamma_summary_df

```

```

Out[31]:

```

	experiment	alpha	gamma	epsilon_decay	step_reward	invalid_move_reward	hazar
0	gamma_0.50	0.5	0.50	0.995	-1.0	-5.0	
1	gamma_0.75	0.5	0.75	0.995	-1.0	-5.0	
2	gamma_0.95	0.5	0.95	0.995	-1.0	-5.0	
3	gamma_0.99	0.5	0.99	0.995	-1.0	-5.0	

The discount-factor table compares the final performance of each gamma value. Since the delivery reward is delayed, I expect higher gamma values to perform better because they preserve the value of future rewards during Q-value updates.

```

In [32]: plt.figure(figsize=(10, 5))

for history, _ in gamma_results:
    plt.plot(
        history["episode"],
        history["rolling_return"],
        label=history["experiment"].iloc[0],
    )

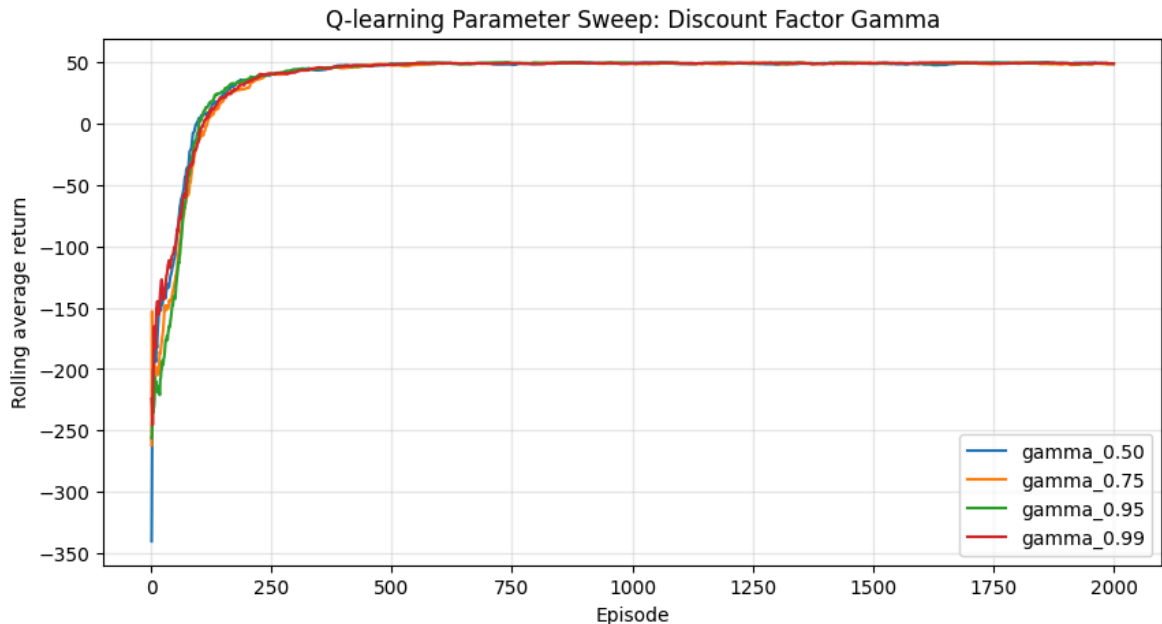
plt.xlabel("Episode")
plt.ylabel("Rolling average return")
plt.title("Q-learning Parameter Sweep: Discount Factor Gamma")

```

```
plt.legend()
plt.grid(True, alpha=0.3)

gamma_plot_path = RESULTS_DIR / "parameter_sweep_gamma.png"
plt.savefig(gamma_plot_path, dpi=300, bbox_inches="tight")
plt.show()

print(f"Saved plot to: {gamma_plot_path}")
```



Saved plot to: ..\Results\basic\parameter_sweep_gamma.png

5.3 Epsilon Decay Experiments

The epsilon decay rate controls how quickly the agent shifts from exploration to exploitation. If epsilon decays too quickly, the agent may exploit an incomplete Q-table. If it decays too slowly, the agent may continue taking unnecessary random actions even after a good policy has been learned.

```
In [33]: epsilon_decay_experiments = [
    ("fast_decay_0.980", 0.980),
    ("medium_decay_0.995", 0.995),
    ("slow_decay_0.999", 0.999),
]

epsilon_results = []

for name, decay_value in epsilon_decay_experiments:
    _, history, summary = run_q_learning_experiment(
        experiment_name=name,
        config=config,
        episodes=2000,
        alpha=0.5,
        gamma=0.95,
        epsilon_start=1.0,
        epsilon_min=0.05,
        epsilon_decay=decay_value,
        eval_every=50,
        eval_episodes=20,
    )
```

```

    history = add_rolling_metrics(history, window=50)
    history["experiment"] = name

    epsilon_results.append((history, summary))

epsilon_summary_df = pd.DataFrame([summary for _, summary in epsilon_results]).reset_index()
epsilon_summary_df

```

Out[33]:

	experiment	alpha	gamma	epsilon_decay	step_reward	invalid_move_reward
0	fast_decay_0.980	0.5	0.95	0.980	-1.0	-5.0
1	medium_decay_0.995	0.5	0.95	0.995	-1.0	-5.0
2	slow_decay_0.999	0.5	0.95	0.999	-1.0	-5.0

The epsilon-decay table compares the effect of different exploration schedules. The greedy evaluation metrics show the final learned policy, while the final training metrics also reflect the amount of random exploration still occurring near the end of training.

In [34]:

```

plt.figure(figsize=(10, 5))

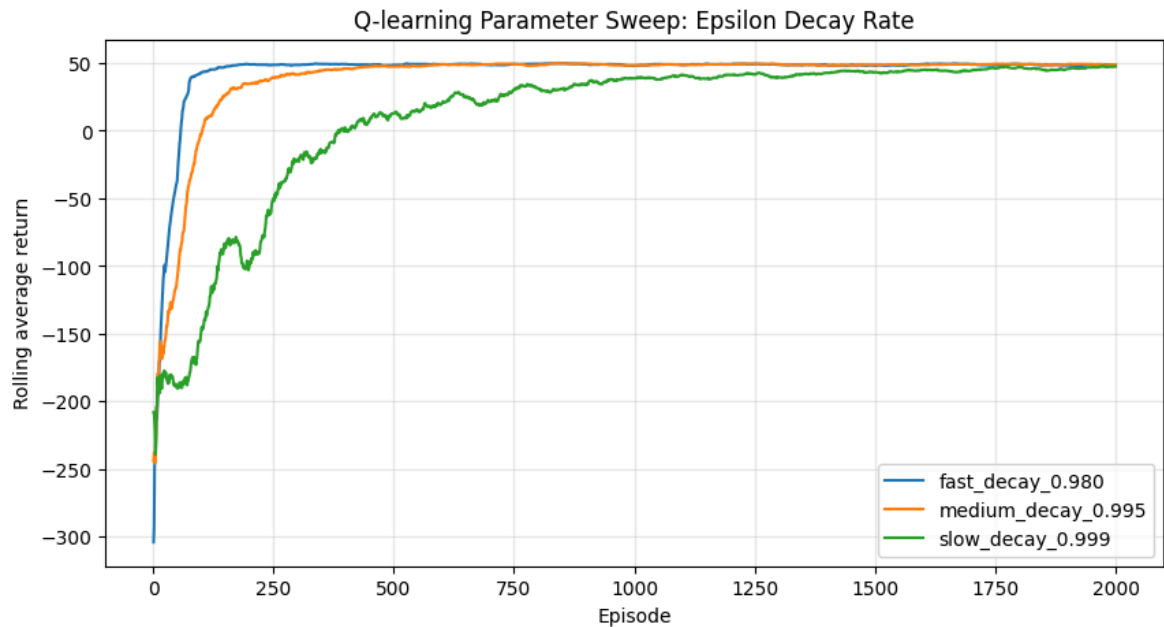
for history, _ in epsilon_results:
    plt.plot(
        history["episode"],
        history["rolling_return"],
        label=history["experiment"].iloc[0],
    )

plt.xlabel("Episode")
plt.ylabel("Rolling average return")
plt.title("Q-learning Parameter Sweep: Epsilon Decay Rate")
plt.legend()
plt.grid(True, alpha=0.3)

epsilon_plot_path = RESULTS_DIR / "parameter_sweep_epsilon_decay.png"
plt.savefig(epsilon_plot_path, dpi=300, bbox_inches="tight")
plt.show()

print(f"Saved plot to: {epsilon_plot_path}")

```



Saved plot to: ..\Results\basic\parameter_sweep_epsilon_decay.png

The epsilon-decay curve shows the exploration-exploitation trade-off. Fast decay can improve training return quickly if enough useful states have already been explored, while slow decay can delay stable performance because the agent continues to take exploratory actions for longer.

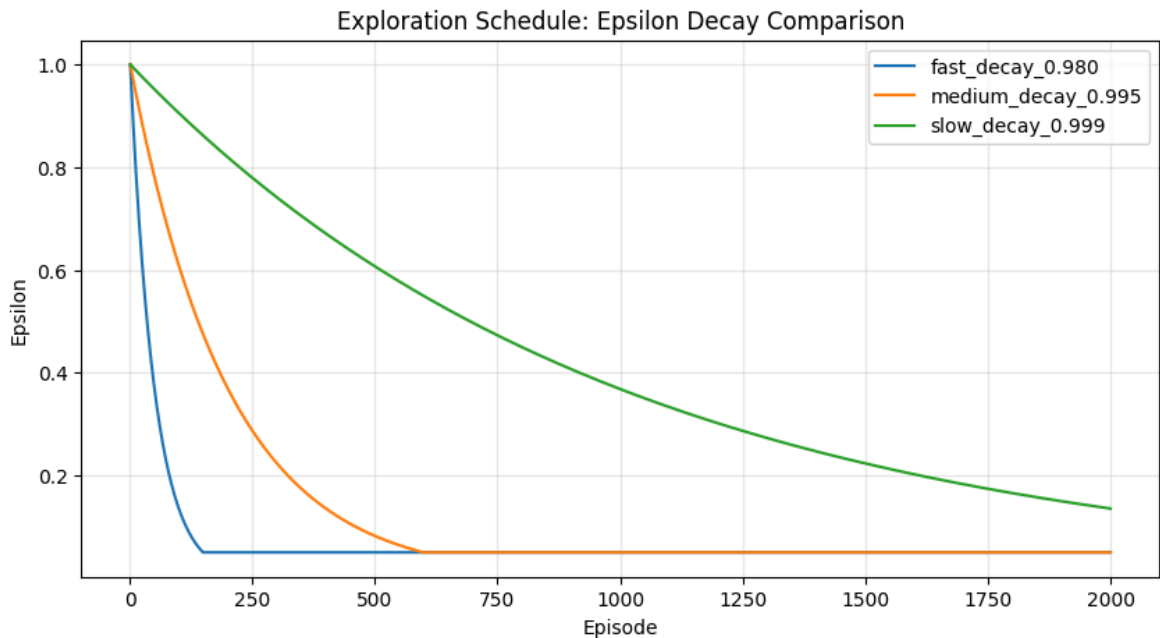
```
In [35]: plt.figure(figsize=(10, 5))

for history, _ in epsilon_results:
    plt.plot(
        history["episode"],
        history["epsilon"],
        label=history["experiment"].iloc[0],
    )

plt.xlabel("Episode")
plt.ylabel("Epsilon")
plt.title("Exploration Schedule: Epsilon Decay Comparison")
plt.legend()
plt.grid(True, alpha=0.3)

epsilon_schedule_plot_path = RESULTS_DIR / "epsilon_decay_schedules.png"
plt.savefig(epsilon_schedule_plot_path, dpi=300, bbox_inches="tight")
plt.show()

print(f"Saved plot to: {epsilon_schedule_plot_path}")
```



Saved plot to: `..\Results\basic\epsilon_decay_schedules.png`

This plot shows the actual exploration schedule used in each epsilon-decay experiment. It helps explain differences between the training curves because higher epsilon values correspond to more random exploratory actions.

5.2.1 Interpretation of Discount Factor Results

All gamma settings reach a final greedy evaluation return of 50.0, a mean greedy evaluation length of 10.0 steps, and a success rate of 1.0. This shows that the final learned policy is robust to the discount factor in this environment.

The limited difference between gamma settings is explained by the small deterministic state space and shaped reward function. The agent receives useful feedback from the step penalty, pickup reward and delivery reward, so even lower gamma values can still learn a successful route. In a larger or sparser environment, I would expect gamma to have a stronger effect because delayed rewards would be harder to propagate back to earlier states.

5.3.1 Interpretation of Epsilon Decay Results

The epsilon decay experiments show a clearer difference than the alpha and gamma experiments. All three settings eventually learn a successful greedy policy, but they differ in convergence speed and late-training behaviour.

The fast decay setting reaches high rolling return quickly because the agent shifts rapidly from exploration to exploitation. The medium decay setting is slightly slower but still converges efficiently. The slow decay setting learns much more gradually because epsilon remains high for longer, meaning the agent continues to take random exploratory actions even after useful Q-values have been learned.

The final greedy evaluation is identical across all three settings, but the training curves differ because training performance includes exploration. This demonstrates why it is

important to report both training curves and greedy-policy evaluation.

5.4 Reward Design Experiments

Reward design is a central part of reinforcement learning because the reward function defines the behaviour being optimised. I compare the baseline shaped reward function against alternative reward schemes to evaluate how reward shaping affects learning.

The reward schemes are:

- sparse reward: only successful delivery is strongly rewarded,
- baseline shaped reward: step cost, invalid-move penalty, hazard penalty, pickup reward and delivery reward,
- harsh penalty reward: stronger penalties for invalid and hazardous movement,
- weak delivery reward: reduced final reward, making the goal signal less dominant.

```
In [36]: reward_configs = {
    "sparse_reward": GridWorldConfig(
        step_reward=0.0,
        invalid_move_reward=0.0,
        hazard_reward=0.0,
        pickup_reward=0.0,
        delivery_reward=50.0,
    ),
    "baseline_shaped": config,
    "harsh_penalties": GridWorldConfig(
        step_reward=-1.0,
        invalid_move_reward=-15.0,
        hazard_reward=-25.0,
        pickup_reward=10.0,
        delivery_reward=50.0,
    ),
    "weak_delivery": GridWorldConfig(
        step_reward=-1.0,
        invalid_move_reward=-5.0,
        hazard_reward=-10.0,
        pickup_reward=10.0,
        delivery_reward=20.0,
    ),
}

reward_results = []

for name, reward_config in reward_configs.items():
    _, history, summary = run_q_learning_experiment(
        experiment_name=name,
        config=reward_config,
        episodes=2000,
        alpha=0.5,
        gamma=0.95,
        epsilon_start=1.0,
        epsilon_min=0.05,
        epsilon_decay=0.995,
        eval_every=50,
        eval_episodes=20,
```

```

    )

    history = add_rolling_metrics(history, window=50)
    history["experiment"] = name

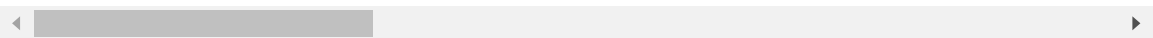
    reward_results.append((history, summary))

reward_summary_df = pd.DataFrame([summary for _, summary in reward_results]).rou
reward_summary_df

```

Out[36]:

	experiment	alpha	gamma	epsilon_decay	step_reward	invalid_move_reward	h
0	sparse_reward	0.5	0.95	0.995	0.0	0.0	
1	baseline_shaped	0.5	0.95	0.995	-1.0	-5.0	
2	harsh_penalties	0.5	0.95	0.995	-1.0	-15.0	
3	weak_delivery	0.5	0.95	0.995	-1.0	-5.0	



The reward-design table compares final performance under different reward functions. The greedy evaluation metrics show whether the final learned policy completes the task, while the training metrics show how easy or difficult each reward scheme made learning.

```

In [37]: plt.figure(figsize=(10, 5))

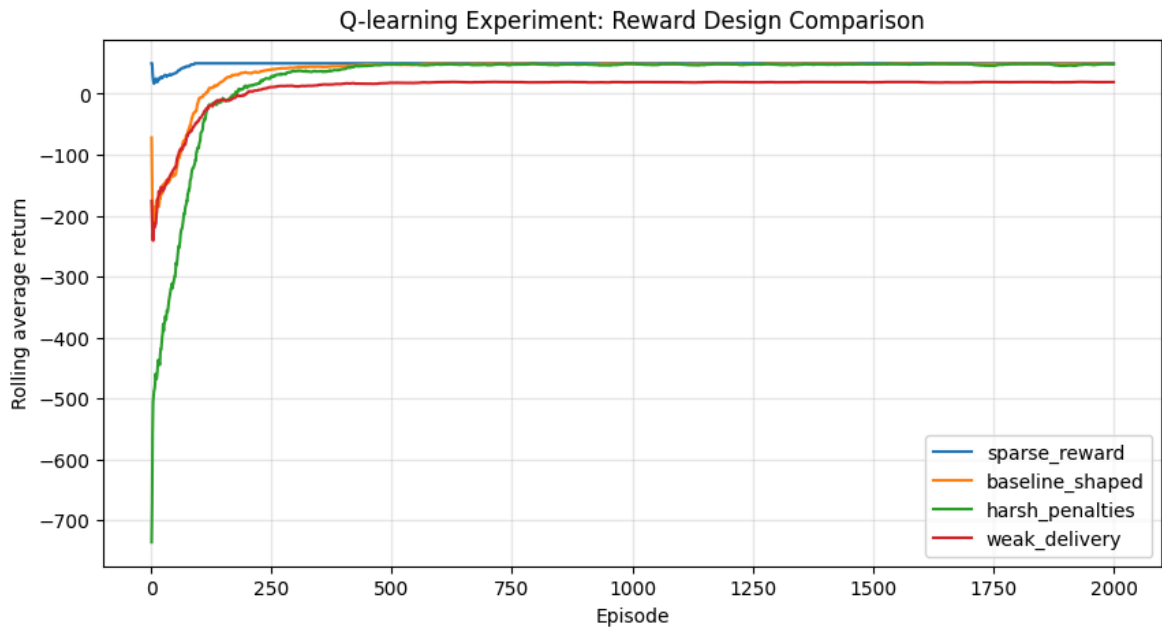
for history, _ in reward_results:
    plt.plot(
        history["episode"],
        history["rolling_return"],
        label=history["experiment"].iloc[0],
    )

plt.xlabel("Episode")
plt.ylabel("Rolling average return")
plt.title("Q-learning Experiment: Reward Design Comparison")
plt.legend()
plt.grid(True, alpha=0.3)

reward_plot_path = RESULTS_DIR / "reward_design_comparison.png"
plt.savefig(reward_plot_path, dpi=300, bbox_inches="tight")
plt.show()

print(f"Saved plot to: {reward_plot_path}")

```



Saved plot to: ..\Results\basic\reward_design_comparison.png

The reward-design curve shows how the structure of the reward function affects learning. Reward shaping can speed up learning by giving the agent useful intermediate feedback, while sparse or poorly scaled rewards can make the learning signal less informative.

```
In [38]: reward_eval_columns = [
    "experiment",
    "step_reward",
    "invalid_move_reward",
    "hazard_reward",
    "pickup_reward",
    "delivery_reward",
    "mean_training_return_last_100",
    "mean_training_steps_last_100",
    "greedy_eval_mean_return",
    "greedy_eval_mean_steps",
    "greedy_eval_success_rate",
]

reward_summary_df[reward_eval_columns]
```

```
Out[38]:
```

	experiment	step_reward	invalid_move_reward	hazard_reward	pickup_reward	delivery_reward
0	sparse_reward	0.0	0.0	0.0	0.0	0.0
1	baseline_shaped	-1.0	-5.0	-10.0	10.0	10.0
2	harsh_penalties	-1.0	-15.0	-25.0	10.0	10.0
3	weak_delivery	-1.0	-5.0	-10.0	10.0	10.0

This compact table keeps only the most relevant reward-design comparison columns. It is suitable for use in the written report because it links each reward scheme directly to the final performance metrics.

5.4.1 Interpretation of Reward Design Results

All reward schemes eventually learn a successful greedy policy with a success rate of 1.0. However, the reward-design curves show that the learning process is strongly affected by the structure and scale of the reward function.

The sparse reward setting reaches a final greedy return of 50.0 because there is no step penalty, so any successful delivery receives the full delivery reward. However, this does not mean it is a better reward function: without a step cost, the return does not strongly distinguish between shorter and longer successful routes during training.

The baseline shaped reward also reaches a greedy return of 50.0, but it does so while encouraging efficient behaviour through a step cost, pickup reward and penalties for poor movement. This makes the learned behaviour more meaningful because the agent is rewarded for completing the task efficiently rather than merely completing it.

The harsh-penalty setting begins with much lower returns because exploratory actions are punished more severely when the agent hits invalid moves or hazards. It still converges to the same final greedy policy, showing that Q-learning can recover from severe early penalties in this small deterministic environment.

The weak-delivery setting also learns a successful policy, but its final greedy return is only 20.0 because the delivery reward is smaller. This confirms that reward scale directly affects the reported return even when the learned route is equally successful.

5.5 Overall Parameter Experiment Summary

I combine the main experiment summaries into one table so that the effects of alpha, gamma, epsilon decay and reward design can be compared directly. The table focuses on final greedy-policy performance and late-stage training performance.

```
In [39]: combined_experiment_summary = pd.concat(
    [
        alpha_summary_df.assign(category="alpha"),
        gamma_summary_df.assign(category="gamma"),
        epsilon_summary_df.assign(category="epsilon_decay"),
        reward_summary_df.assign(category="reward_design"),
    ],
    ignore_index=True,
)

combined_columns = [
    "category",
    "experiment",
    "alpha",
    "gamma",
    "epsilon_decay",
    "mean_training_return_last_100",
    "mean_training_steps_last_100",
    "final_epsilon",
    "greedy_eval_mean_return",
    "greedy_eval_mean_steps",
    "greedy_eval_success_rate",
]
```

```
combined_experiment_summary_compact = combined_experiment_summary[combined_column]
combined_experiment_summary_compact
```

Out[39]:

	category	experiment	alpha	gamma	epsilon_decay	mean_training_return
0	alpha	alpha_0.1	0.1	0.95	0.995	
1	alpha	alpha_0.5	0.5	0.95	0.995	
2	alpha	alpha_0.9	0.9	0.95	0.995	
3	gamma	gamma_0.50	0.5	0.50	0.995	
4	gamma	gamma_0.75	0.5	0.75	0.995	
5	gamma	gamma_0.95	0.5	0.95	0.995	
6	gamma	gamma_0.99	0.5	0.99	0.995	
7	epsilon_decay	fast_decay_0.980	0.5	0.95	0.980	
8	epsilon_decay	medium_decay_0.995	0.5	0.95	0.995	
9	epsilon_decay	slow_decay_0.999	0.5	0.95	0.999	
10	reward_design	sparse_reward	0.5	0.95	0.995	
11	reward_design	baseline_shaped	0.5	0.95	0.995	
12	reward_design	harsh_penalties	0.5	0.95	0.995	
13	reward_design	weak_delivery	0.5	0.95	0.995	

This combined table shows that all tested configurations eventually produce a successful greedy policy in this environment. The main differences are therefore in learning speed, training stability and reward scale rather than final task completion.

```
In [40]: combined_summary_path = RESULTS_DIR / "q_learning_experiment_summary.csv"
combined_experiment_summary_compact.to_csv(combined_summary_path, index=False)

print(f"Saved combined experiment summary to: {combined_summary_path}")
```

Saved combined experiment summary to: ..\Results\basic\q_learning_experiment_summary.csv

I save the combined experiment summary as a CSV file so that the numerical results can be reused in the written report without manually copying values from notebook outputs.

Task 6: Quantitative and Qualitative Analysis

The Q-learning experiments show that the custom GridWorld task is solved reliably under a range of parameter settings. Quantitatively, the strongest evidence is the final greedy-policy performance: the baseline agent achieves a mean evaluation return of 50.0, a mean evaluation length of 10.0 steps, and a success rate of 1.0. Qualitatively, the learned

greedy rollout shows that the agent follows a sensible route: it moves toward the pickup cell, collects the package, and then travels directly to the delivery cell while avoiding blocked and hazardous cells.

6.1 Quantitative Analysis

The baseline training curve shows clear improvement over time. Early episodes have strongly negative returns because the agent explores randomly and often takes inefficient or invalid actions. As Q-values are updated, the rolling average return increases and the rolling average episode length decreases. This indicates that the agent learns both to complete the task and to complete it in fewer steps.

The final baseline training performance is slightly lower than the greedy evaluation performance because training still includes exploration at ($\epsilon = 0.05$). This means that even late in training, some actions are random and may produce unnecessary steps or penalties. The greedy evaluation removes this exploration and therefore gives a cleaner estimate of the learned policy.

Across the alpha and gamma sweeps, all tested settings eventually reached a greedy evaluation success rate of 1.0. This suggests that the environment is small and well-shaped enough for Q-learning to converge reliably. The epsilon-decay experiment produced the clearest difference in learning speed: fast decay reached high return quickly, medium decay converged reliably, and slow decay delayed convergence because the agent continued exploring for longer.

The reward-design experiment showed that final success can hide important differences in the learning process. Sparse reward, baseline shaped reward, harsh penalties and weak delivery reward all solved the task, but their return curves differed substantially. This confirms that reward design affects training dynamics even when the final greedy policy is successful.

6.2 Qualitative Analysis

The learned behaviour is consistent with the intended objective. The agent does not simply maximise immediate reward; it learns the correct sequence of sub-goals required by the task. It first navigates to the pickup location while not carrying the package, then changes to the carrying state, and finally moves to the delivery location. This confirms that including the carrying variable in the state representation is necessary because the same grid position can have different meanings depending on whether the package has already been collected.

The shaped reward function helps the agent learn efficient behaviour. The step cost discourages wandering, the invalid-move penalty discourages repeatedly attempting blocked movements, and the hazard penalty discourages unsafe routes. The pickup reward provides an intermediate learning signal before the final delivery reward, which is useful because delivery is only possible after the package has been collected.

The results also show the difference between learning performance and final policy performance. Training curves include exploratory actions, so they remain slightly noisy even after the policy has effectively converged. Greedy evaluation is therefore essential for judging the final learned policy.

6.3 Limitations of the Basic Q-learning Approach

Although tabular Q-learning works well in this GridWorld, it does not scale well to large or continuous state spaces. The Q-table requires one value for every state-action pair, so the table grows quickly as the environment becomes larger or the state representation becomes more detailed.

The environment is also deterministic and fully observable. This makes it suitable for demonstrating Q-learning, but it is simpler than many real reinforcement learning problems. In stochastic or partially observable environments, learning would likely require more episodes, more careful exploration, and potentially function approximation. This motivates the advanced part of the coursework, where I use deep reinforcement learning methods to handle continuous observation spaces.

```
In [41]: np.save(RESULTS_DIR / "baseline_q_table.npy", baseline_q_table)
baseline_history.to_csv(RESULTS_DIR / "baseline_q_learning_history.csv", index=False)
greedy_rollout_df.to_csv(RESULTS_DIR / "baseline_greedy_rollout.csv", index=False)

print("Saved baseline Q-table, training history and greedy rollout.")
```

Saved baseline Q-table, training history and greedy rollout.

The main outputs from the basic Q-learning notebook are saved for reproducibility. These files include the learned Q-table, the baseline training history and the final greedy rollout.